UPNOW · FRONTEND ASSESSMENT

Integrate login, listings & stats

Build three working screens in Next.js against the live UpNow staging API. Read this through before you start — section 1 contains a setup step you cannot skip, and the contracts in section 5 include details that are easy to miss.

1 Getting set up

REPOSITORY	`https://github.com/UpNowUAE/upnow-frontend-exercise`
API BASE	`https://staging.upnow.ae/api`
API DOCS	`API.md` — in the repository root

You will receive an invitation to a private GitHub repository. It is a blank Next.js 16 app — App Router, TypeScript, Tailwind 4 — with the UpNow brand tokens already defined in `src/app/globals.css`, so utilities like `bg-primary`, `text-neutral-500` and `bg-surface` work immediately.

```
git clone <your repo url>
cp .env.local.example .env.local
npm install
npm run dev          # http://localhost:3000
```

Nothing else is set up — no data layer, no components, no fetch wrapper. Putting those in place is most of the exercise.

> **Do this before anything talks to the API.** Calling staging directly from the browser is blocked by CORS — that is expected, not a broken environment. Set up the Next.js rewrite proxy in section 5.1 first.

2 Screen one — public listings

- Fetch units from `GET /listings/units` and render them as cards.
- Pagination or infinite scroll — your choice.
- At least one working filter (search, emirate or price) driven by the API's own query params, **not** by filtering an already-fetched array in the browser.
- Loading, empty and error states. The API can be slow and it can fail; both should look deliberate rather than broken.
- A unit detail view from `GET /listings/units/:id`.

3 Screen two — login

- The two-step OTP flow: email → code → token.
- Validation and error handling on both steps — wrong code, expired code, unknown email, rate-limited.
- Persist the session and show something that proves it worked, for example the signed-in user from `GET /auth/me`.

Where you keep the token is your call. Make one, and be ready to explain it.

4 Screen three — stats dashboard

- Pull portfolio numbers from `GET /reports/executive-summary` and render them as headline stat cards.
- Add one more view from a second stats endpoint — occupancy, upcoming renewals or vacancy. A chart is welcome, but a well-built table is equally fine.
- These endpoints need the token from screen two, so this screen only works once login does. Handle the signed-out case rather than letting it throw.

5 API reference

5.1 · Running against staging from localhost

The API only accepts browser requests from origins on its allow-list, and `http://localhost:3000` is **not** on it. A `fetch()` straight from your React components to `https://staging.upnow.ae/api/...` will be blocked.

Do not work around this with a browser extension or a `no-cors` fetch. Proxy through your own Next.js server — same-origin from the browser's point of view, and it is how the real UpNow frontend is built:

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  async rewrites() {
    return [
      {
        source: "/api/:path*",
        destination: "https://staging.upnow.ae/api/:path*",
      },
    ];
  },
};

export default nextConfig;
```

```
# .env.local
NEXT_PUBLIC_API_URL=/api
```

Now call `/api/listings/units` from your app — no absolute URL, no CORS, and the `Authorization` header passes straight through. Restart `npm run dev` after editing `next.config.ts`; rewrites are not hot-reloaded.

5.2 · Endpoints

Public listings — no authentication

These endpoints are world-readable. Start here; the entire listings screen can be built without logging in.

`GET /listings/units` — paginated available units.

QUERY PARAM	NOTES
`page`	default `1`
`limit`	default `20`
`q`	free-text search
`category`	space category
`spaceType`	
`bedrooms`	
`minPrice` / `maxPrice`	
`emirate`	e.g. `DXB`
`country`	ISO-3166 alpha-2; defaults from the caller's region
`sort`	

Response envelope:

```
{
  "items": [
    {
      "unit": {
        "id": 112, "unitNumber": "LAND-1", "unitName": "...",
        "unitType": "commercial_land", "floor": "0",
        "propertyId": 5, "property": { /* nested */ }
      },
      "property": {
        "id": 5, "buildingName": "Marble Factory",
        "fullAddress": "16B St, Ras Al Khor", "spaceCategory": "land",
        "emirate":   { "code": "DXB", "nameEn": "Dubai" },
        "community": { "nameEn": "Ras Al Khor" },
        "latitude": "25.1785584", "longitude": "55.3574528"
      },
      "coverPhotoUrl": "https://..."
    }
  ],
```

```
  "total": 0, "page": 1, "pageSize": 20
}
```

> **Note.** Both `unit.property` and the top-level `property` exist on every row. Decide which one your components read from, and be consistent about it.

ENDPOINT	RETURNS
`GET /listings/units/:id`	One unit's full detail
`GET /listings/locations`	Filter options. Optional `country`, `emirate`, `category`
`GET /listings/floor-plans`	Optional `propertyId`, `bedrooms`

Login — two steps

> **Read this twice.** Step 1 takes `email`. Step 2 takes `identifier` and `code`. Sending `email` to step 2 fails validation — the field name genuinely changes between the two calls.

Step 1 — request the code

```
POST /auth/login
Content-Type: application/json

{ "email": "someone@example.com" }
```

```
{
  "requiresVerification": true,
  "verificationStep": "email",          // or "phone"
  "maskedPhone": "s.....e@e.....e.com",
  "devOtp": "482915"                    // staging only
}
```

> **On `devOtp`.** Staging returns the code in the response so you can complete the flow without access to a mailbox. It is **absent in production.** Do not build anything that depends on it — your login UI must still work when that field is missing.

Step 2 — exchange the code for a token

```
POST /auth/login/verify-otp
Content-Type: application/json

{ "identifier": "someone@example.com", "code": "482915" }
```

```
{ "access_token": "eyJhbGciOi...", "user": { "id": 1, "role": "rental_provider" } }
```

Step 3 — use it

```
GET /auth/me
Authorization: Bearer <access_token>
```

Stats endpoints — authentication required

Each needs `Authorization: Bearer <access_token>` and returns data scoped to the signed-in provider. Omit the header and you get a `401`.

ENDPOINT	RETURNS
`GET /reports/executive-summary`	Headline KPIs plus cashflow, renewal, occupancy and aging chart series. The best single source for a dashboard. Optional `year`, defaults to the current year.
`GET /reports/occupancy`	Occupancy snapshot — units, occupied counts and contracted rent per property. Optional `occupancy`: `occupied` / `vacant` / `all`.
`GET /reports/lease-expiry`	Leases coming up for renewal, grouped by month. Optional `window` in days, default `90`.
`GET /reports/vacancy-analysis`	Vacant units to action — asking rent at stake and listing status.
`GET /reports/portfolio-overview`	Unit mix — a row per property × unit type × bedrooms × layout × size.

Shared optional filters on most of the above: `buildings` (comma-separated property ids), `leaseTypes` (`direct`, `sub`, `management`) and `year` where the report is year-bound.

> These return real shapes with real nesting. Inspect a response before you model it — do not guess the field names.

Rate limits

These are real and easy to trip while iterating:

- **100 requests / 60 seconds per IP**, globally. A component that refetches on every render will hit this, and the `429` looks exactly like a broken account.
- **10 requests / 10 minutes per IP** on `POST /auth/login/verify-otp`.

Test account. Sent to you separately.

6 What we are looking at

AREA	WHAT WE LOOK FOR
API integration	Where fetching lives, how errors and loading are handled, whether the response shape is typed rather than `any`
App Router use	Server components by default; `"use client"` only where interactivity genuinely needs it
Component boundaries	Sensible decomposition — no 400-line page files
Resilience	Respecting the rate limits above; no refetch loops
Responsive layout	It should hold up on a phone as well as a laptop
Readable code	Consistent naming, no dead code or stray console logs

7 Ground rules & handing it back

- Any library you would normally reach for is fine — add it.
- Documentation, Google and AI assistants are all allowed. We are interested in how you work, not what you have memorised.
- Ambiguity is expected. Make a call, note it, and move on.

Commit and push to the repository we shared with you. Include a short `NOTES.md` covering what you finished, what you would do next, and anything you deliberately skipped and why. Explaining a trade-off counts in your favour.